

CÁLCULO EN MOVIMIENTO

Arquitectura de Software, Motor Matemático Simbólico, Visión Artificial por Gestos y Visualizador 3D Reactivo

Componente	Especificación
Nombre del Software	Cálculo en Movimiento (Proyecto Prima)
Tipo de Aplicación	Web Full-Stack Interactiva Científica (3D + Visión)
Backend	Python 3.12, FastAPI, SymPy, NumPy, OpenCV, MediaPipe
Frontend	React 18, TypeScript, Three.js, Vite, Lucide Icons
Protocolos	REST API (HTTP/JSON) y WebSocket (/ws/vision)
Modos de Entrada	Modo Cámara (visión de 1 a 5 dedos) y Modo Manual
Fecha de Publicación	07 de October de 2026
Estado Operativo	ACTIVO Y EN PRODUCCIÓN LOCAL

Resumen de Propósito: Esta aplicación trasciende las calculadoras gráficas tradicionales al conectar de forma bidireccional el análisis formal de cálculo diferencial con una representación volumétrica 3D inmersiva y un sistema de control gestual basado en visión artificial. El usuario puede ingresar funciones matemáticas de cualquier familia o utilizar su mano para transitar entre los 5 requisitos fundamentales del cálculo diferencial en tiempo real.

1. Visión General del Proyecto

Cálculo en Movimiento es una suite pedagógica y de ingeniería diseñada para materializar visualmente los conceptos abstractos del cálculo diferencial. La aplicación resuelve la brecha entre la definición simbólica rigurosa y su manifestación geométrica espacial.

El sistema ofrece dos modalidades de operación perfectamente orquestadas:

- **Modo Cámara:** Permite al usuario interactuar sin periféricos físicos. Al colocar la mano frente a la cámara web, MediaPipe y OpenCV reconocen los dedos extendidos (1 a 5) y conmutan automáticamente entre los 5 módulos temáticos del cálculo.
- **Modo Manual:** Permite un control fino mediante un selector de dedos interactivo en pantalla, controles deslizantes para parámetros como la abscisa x o el intervalo $[a, b]$, y un teclado científico de funciones para ingresar cualquier expresión algebraica.

2. Los 5 Requisitos Fundamentales del Cálculo

Dedo	Requisito	Concepto Matemático y Visualización 3D
1	Evaluación y Pendiente	Evalúa $f(x)$, calcula $f'(x)$ simbólicamente y grafica la Recta Tangente exacta. Dibuja un triángulo de pendiente $(\Delta y/\Delta x)$, línea de proyección vertical al eje X y badge holográfico con la ecuación punto-pendiente.
2	Secante y Razón de Cambio	Traza la Recta Secante entre dos puntos $A(a, f(a))$ y $B(b, f(b))$. Calcula la razón de cambio promedio $[f(b)-f(a)]/(b-a)$ e ilustra el proceso de paso al límite cuando Δx tiende a 0.
3	Función y Derivadas	Grafica simultáneamente en planos paralelos en el eje Z: la función base $f(x)$ en $z=0$, la primera derivada $f'(x)$ en $z=1.5$ y la segunda derivada $f''(x)$ en $z=3.0$, permitiendo comparar visualmente concavidades y pendientes.
4	Puntos Críticos	Encuentra los ceros de la derivada $f'(x)=0$ mediante el Teorema de Fermat . Clasifica cada punto crítico como Máximo Local, Mínimo Local o Punto de Inflexión utilizando el criterio de la segunda derivada $f''(c)$.
5	Desafío Aplicado	Modela un problema físico o de optimización real (como la cinemática de una partícula en movimiento: posición $s(t)$, velocidad instantánea $v(t) = s'(t)$ y aceleración $a(t) = s''(t)$).

3. Stack Tecnológico Detallado

Capa	Tecnología	Rol y Responsabilidad en el Sistema
Frontend Core	React 18 + TypeScript	Gestión del árbol de componentes, tipado estricto de datos matemáticos y de telemetría de visión, estado reactivo sincronizado.
Gráficos 3D	Three.js (WebGL)	Renderizado volumétrico con TubeGeometry, CatmullRomCurve3, materiales metálicos, sistema de iluminación ACESFilmicToneMapping, sprites de texto en alta resolución y cámaras orbitales.
Bundler / Servidor	Vite 5	Compilación y empaquetado de ultra-alta velocidad, Hot Module Replacement (HMR) y servidor de desarrollo en puerto 5173.
Estilos UI	Vanilla CSS Modular	Variables de diseño (design tokens), tipografías monoespaciadas para fórmulas, modo oscuro técnico y glassmorphism.
Backend API	FastAPI (Python)	Framework asíncrono de alto rendimiento, validación automática con Pydantic, endpoints REST y servidor WebSocket en puerto 8000.
Cálculo Simbólico	SymPy	Diferenciación exacta, resolución analítica de ecuaciones $f'(x)=0$, simplificación algebraica de tangentes y secantes, conversión a LaTeX.
Cálculo Numérico	NumPy	Discretización adaptativa de dominios $[x_min, x_max]$ con generación de mallas de 200 a 400 puntos 3D libres de discontinuidades.
Visión Artificial	MediaPipe + OpenCV	HandLandmarker con 21 keypoints anatómicos en 3D, inferencia en tiempo real sobre frames de webcam, buffer circular de estabilización y conteo de dedos.

4. Diagrama de Flujo y Comunicación del Sistema

La interacción entre capas sigue un modelo de desacoplamiento estricto:

- **1. Entrada del Usuario:** El usuario escribe una función matemática en el panel izquierdo o la selecciona de la biblioteca de ejemplos (cuadráticas, trigonométricas, exponenciales, etc.).
- **2. Validación Inmediata:** El frontend envía la expresión a `/api/validate`. El parser limpia sintaxis (como `4x` a `4*x`), comprueba que sea matemáticamente analizable y clasifica su tipo sin calcular gráficas aún.
- **3. Selección de Modo:** Si se selecciona *Modo Cámara*, el backend enciende la webcam a través de DirectShow (índice 0) y transmite el video con landmarks dibujados por WebSocket (`/ws/vision`). Al detectar un número de dedos confirmado, se dispara el cálculo automático. En *Modo Manual*, el usuario pulsa directamente los botones de 1 a 5 dedos.
- **4. Cálculo y Generación Geométrica:** El motor en `/api/calculate` evalúa la derivada, la recta tangente o secante, los puntos críticos y genera los arrays de coordenadas tridimensionales (x, y, z).
- **5. Renderizado 3D Reactivo:** Three.js recibe el payload JSON y actualiza la escena: traza el tubo de la curva en cian brillante, la recta en verde esmeralda, ubica el punto P con halo animado, añade el triángulo de pendiente y despliega badges de alta legibilidad.

5. El Motor Matemático Simbólico (``backend/math``)

El núcleo matemático fue construido siguiendo los más rigurosos estándares de rigor algebraico y cálculo analítico exacto:

A. Parser Matemático Inteligente (``parser.py``):

Los usuarios ingresan expresiones de múltiples formas coloquiales. El parser implementa transformaciones léxicas antes de la compilación simbólica:

- **Multiplicación implícita:** Convierte patrones como $4x$, $2(x+1)$ o $x \sin(x)$ en $4*x$, $2*(x+1)$ y $x*\sin(x)$ mediante expresiones regulares precisas.
- **Manejo de potencias:** Convierte sintaxis x^2 en $x**2$ entendible por Python.
- **Constantes naturales:** Sustituye e^x por $\exp(x)$ preservando el número de Euler analítico.
- **Clasificación automática:** Clasifica la función en: Constante, Lineal, Cuadrática, Cúbica, Polinómica, Racional, Raíz, Trigonométrica, Exponencial o Logarítmica.
- **Seguridad (Sandbox):** Prohíbe llamadas a funciones peligrosas o builtins de Python, permitiendo únicamente el conjunto matemático de SymPy.

B. Motor de Ejecución de Requisitos (``engine.py``):

Cada uno de los 5 requisitos cuenta con un ejecutor matemático dedicado:

- **Requisito 1 ($f'(a)$ y Recta Tangente):** Utiliza diferenciación simbólica exacta $\text{diff}(f, x)$. Evalúa $f(a)$ y la pendiente $m = f'(a)$. Construye la ecuación analítica $y = m*(x - a) + f(a)$ y su forma simplificada $y = m*x + b$.
- **Requisito 2 (Recta Secante y $\Delta x \rightarrow 0$):** Evalúa $A(a, f(a))$ y $B(b, f(b))$. Calcula la razón de cambio promedio $\Delta y/\Delta x = (f(b)-f(a))/(b-a)$ y genera la recta secante extendida.
- **Requisito 3 (Función y Derivadas en 3D):** Obtiene $f'(x)$ y $f''(x)$. Genera puntos con desplazamiento en el eje Z: $f(x)$ en $z=0$, $f'(x)$ en $z=1.5$ y $f''(x)$ en $z=3.0$.
- **Requisito 4 (Puntos Críticos y Criterio de Fermat):** Resuelve analíticamente $\text{solve}(f', x)$ en los reales. Para cada raíz c , calcula $f(c)$ y evalúa $f''(c)$. Si $f''(c) < 0$, clasifica como *Máximo local*; si $f''(c) > 0$, como *Mínimo local*; si $f''(c) = 0$, analiza cambio de signo para puntos de inflexión.
- **Requisito 5 (Cinemática Aplicada):** Trata a x como el tiempo t , evaluando posición $s(t)$, velocidad $v(t) = s'(t)$ y aceleración $a(t) = s''(t)$.
- **Muestreo Discreto Seguro (``get_safe_domain``):** Detecta singularidades y asíntotas (por ejemplo en $1/x$ o $\tan(x)$) y segmenta la curva para evitar que Three.js trace líneas que atravesen el infinito.

6. El Módulo de Visión Artificial (`backend/vision`)

La visión artificial implementada permite un control gestual robusto y sin latencia perceptible:

A. Inicialización Directa de Cámara:

En entornos Windows, iterar múltiples índices de cámaras desconocidas genera demoras por timeout de COM en DirectShow. El módulo fue optimizado para abrir de inmediato el dispositivo primario (índice 0) con cv2.CAP_DSHOW en milisegundos, garantizando que la cámara encienda al instante al pulsar 'Encender Cámara'.

B. Modelo MediaPipe HandLandmarker:

El modelo neuronal localiza 21 marcas anatómicas tridimensionales de la mano (muñeca, nudillos MCP, articulaciones PIP/DIP y puntas de dedos TIP). El algoritmo analiza la extensión de cada dedo comparando la coordenada Y de la punta con su respectiva articulación PIP, y la posición relativa del pulgar respecto a la muñeca.

C. Filtro de Estabilización Temporal:

Para evitar que fluctuaciones menores de luz o movimientos rápidos cambien involuntariamente de requisito, se implementó una cola circular (deque de longitud 7) con un umbral de confirmación de 5 lecturas consecutivas idénticas. Solo cuando el gesto es sostenido y estable se envía la orden de cálculo al frontend.

7. El Visualizador 3D con Three.js (`frontend/src/components/Graph3D.tsx`)

La escena 3D fue diseñada con estándares de alta fidelidad visual y contraste óptico:

- **Fondo y Luz de Alto Contraste:** Escena espacial en #070b14 con niebla exponencial sutil e iluminación técnica tri-direccional (Key Light cian, Fill Light violeta y Rim Light esmeralda).
- **Grilla Dual Espacial:** Rejilla principal en $y=0$ con subdivisiones finas, sin planos horizontales opacos que intercepten las curvas con valores negativos.
- **Curvas Volumétricas:** Cada función matemática se renderiza como un tubo 3D brillante (TubeGeometry) con efecto de halo exterior mediante mezcla aditiva.
- **Callout Badges Holográficos:** Las etiquetas de puntos $P(x_{\blacksquare}, f(x_{\blacksquare}))$, rectas tangentes y extremos cuentan con líneas guía punteadas y cuadros con fondo oscuro sólido (rgba(15, 23, 42, 0.95)) y texto blanco nítido para asegurar legibilidad a cualquier distancia o ángulo de rotación.
- **Triángulo de Pendiente y Líneas Guía:** En el Requisito 1 se despliega el triángulo $\Delta y/\Delta x$ que ilustra la razón geométrica de la derivada, junto con la proyección perpendicular hacia el eje X.

8. Historial de Optimizaciones y Correcciones Críticas

Durante el desarrollo y calibración final del proyecto, se diagnosticaron y solucionaron aspectos clave para alcanzar la estabilidad total:

Aspecto Reportado	Causa Raíz Técnica	Solución Implementada
Nombre de Cámara	La interfaz mostraba 'Cámara IA', generando redundancia conceptual.	Se simplificó la denominación en toda la aplicación a ' Cámara ' (y 'Cámara (Activo)').
Encendido de la Cámara	Bucle de prueba sobre 12 combinaciones de índices inexistentes [1, 0, 2, 3] provocaba timeouts de COM en Windows.	Conexión directa inmediata al índice primario 0 con cv2.CAP_DSHOW. Inicio en menos de 200 ms.
Exclusividad de Dedos en Modo Manual	El selector manual de 1 a 5 dedos aparecía también en modo cámara, creando conflicto de control.	Renderizado condicional en App.tsx: el selector manual solo se muestra en Modo Manual. En Modo Cámara se sustituye por una tarjeta de estado de visión.
Contraste y Pantalla Blanca en 3D	Una franja blanca translúcida duplicada a top: 12px en Graph3D.tsx tapaba las tarjetas de telemetría.	Se eliminó la barra duplicada y se consolidó el HUD superior oscuro de 4 tarjetas de alto contraste.
Plano 3D que se veía raro	Existía un PlaneGeometry opaco a y=-0.02 que cortaba las curvas negativas.	Se eliminó el plano y se reemplazó por rejillas vectoriales duales GridHelper limpias.
Integridad Matemática	Requisito estricto de no alterar la lógica matemática analítica.	Se conservaron intactos parser.py y engine.py, validando su suite de 23 pruebas.

9. Guía de Ejecución y Comandos

Para iniciar el entorno completo de desarrollo o producción local, ejecutar:

1. Iniciar Servidor Backend (FastAPI + Visión + SymPy):

```
cd "c:\Users\Fahida\Desktop\proyecto prima"
$env:OPENBLAS_NUM_THREADS="1"; python run_backend.py
```

2. Iniciar Servidor Frontend (React + Vite + Three.js):

```
cd "c:\Users\Fahida\Desktop\proyecto prima\frontend"
npm run dev
```

3. Acceso en el Navegador Web:

- Aplicación Principal: **http://localhost:5173**
- Documentación Interactiva API Swagger: **http://localhost:8000/docs**